Relaytic-AML: A Local-First Agentic Evaluation Lab for Financial-Crime Machine Learning

ML-Enthusiast
Independent researcher
83662706+ML-Enthusiast-de@users.noreply.github.com

June 2026

Abstract

Anti-money laundering (AML) machine-learning results are difficult to trust when privacy constraints, temporal validity, graph provenance, leakage controls, review capacity, and public claims are handled separately. Relaytic-AML is a local-first evaluation lab in which role-scoped agents create auditable evidence cells and conservative claim gates decide what those cells are allowed to support. The current evidence pack reports PaySim synthetic temporal-fraud PR-AUC 0.6388, Elliptic temporal graph-feature PR-AUC 0.6688, and Elliptic2 context PR-AUC 0.9432 +/- 0.0009, with the Elliptic2 row kept below the recorded RevClassifyDS reference of 0.974. The contribution is a reproducible AI evaluation-lab and governance architecture for financial-crime experimentation, not a claim of detector superiority.

1 Introduction

AML modeling is a rare-event, high-stakes evaluation problem. The input may be a stream of mobile-money transfers, card events, account activity, wire messages, customer profiles, or blockchain transactions. Suspicion rarely appears in one row. It often appears as a temporal or network pattern: fast movement of funds after receipt, repeated transfers through related accounts, structured amounts, new counterparties receiving many payments, activity that does not match a customer's profile, or cryptocurrency behavior that involves risky services and geography. Regulatory and typology material from FATF, FinCEN, and FFIEC describes this kind of pattern-based reasoning in operational language [Financial Action Task Force, 2020, Financial Crimes Enforcement Network, 2026, Federal Financial Institutions Examination Council, 2026].

A useful score is not enough. A reader needs to know which data stayed local, which columns were available at decision time, how time or graph boundaries were split, whether model selection touched the test surface, what review budget was assumed, and what interpretation the evidence can support. These questions become sharper when large language models (LLMs) or coding agents assist the research workflow, because fluent explanations can drift away from the artifact record unless the system is designed to fail closed.

Relaytic began as a general local-first inference-engineering lab. Relaytic-AML is the financial-crime edition used here to test whether that architecture can support governed AML experimentation. It is a set of cooperating agents and deterministic harnesses around a local artifact store. The guide helps a user or another agent understand where the run is. The scout checks source posture, schema, leakage risk, and split feasibility. The strategist turns the objective into a task contract. The scientist challenges baselines, ablations, and budget choices. The builder executes bounded runs. Reviewers reconstruct traces. Release governors lint claims, figures, tables, source packages, and public wording against the evidence record.

The paper is therefore a systems and methodology paper. The benchmark rows matter because they exercise the architecture under temporal, graph, operating-point, and claim-governance pressure. They are not presented as a new best detector family. The central question is whether a local evaluation lab can keep evidence, agent assistance, privacy boundaries, and publishable claims aligned.

The work is organized around four research questions:

- **RQ1:** Can Relaytic-AML produce reproducible evidence cells for AML-style temporal and graph tasks?
- **RQ2:** Can it prevent leakage-prone or unsupported claims from being promoted?
- **RQ3:** Can the local artifact record support rowless handoff to external agents while preserving provenance?
- **RQ4:** Do the benchmark rows demonstrate useful, bounded detector evidence under explicit split and budget contracts?

This paper makes four contributions. First, it presents a local-first artifact graph and role-scoped agent runtime for AML evaluation. Second, it defines an evidence-cell schema tying each metric to dataset, split, command, artifact, budget, leakage posture, operating point, and claim state. Third, it contributes a release and claim-gating harness that turns local artifacts into tables, figures, source packages, and manuscript claims while blocking unsupported interpretations. Fourth, it gives public demonstrations on PaySim, Elliptic, and Elliptic2 context rows under strict claim boundaries.

2 Related Work

The AML benchmark landscape is moving toward larger, more realistic, and more graph-native settings. PaySim remains useful as a synthetic mobile-money fraud simulator for temporal proxy experiments [Lopez-Rojas et al., 2016]. Elliptic introduced a public Bitcoin transaction graph with anonymized node features and temporal labels [Weber et al., 2019]. Elliptic2 and RevTrack/RevClassify shifted attention to suspicious subgraphs and richer blockchain context [Bellei et al., 2024, Song et al., 2024]. Recent work such as TransXion, LineMVGNN, quasi-temporal graph extraction, BlazingAML, and continual graph-learning studies shows that the research frontier increasingly treats AML as dynamic graph and systems work rather than static tabular classification [Chen et al., 2026, Poon et al., 2026, Tariq and Hassani, 2026, Ye et al., 2026, Deprez et al., 2025].

Relaytic-AML does not compete with those papers as a new graph-neural detector. Its position is complementary. It asks how an experiment should be governed when data is local or licensed, the task may be temporal or graph-based, analyst review capacity matters, and an agent-assisted workflow must still produce evidence that a skeptical reviewer can audit. That places the system near dataset documentation, model reporting, reproducibility practice, experiment tracking, and governance work [Gebru et al., 2021, Mitchell et al., 2019, Pineau et al., 2021].

The system also differs from adjacent evaluation artifacts. Model cards explain a trained model, but they do not usually bind every number to a command, split, artifact field, and blocked-claim state. Datasheets describe data, but they do not run the model-search and release gates. Reproducibility checklists improve reporting, but they are often static forms rather than executable evidence. MLOps experiment trackers preserve runs, but they are not designed to govern public scientific claims about local, licensed, or privacy-sensitive AML data. Agent benchmarks evaluate agents, while Relaytic-AML uses agents inside an evaluation lab and then tests whether the lab keeps the agents honest. This is the new systems contribution: evidence, agents, local privacy, and publishable wording are coupled in one deterministic release path.

Recent agent-evaluation work shows that language-model systems can produce persuasive research artifacts while still failing on reproduction, validation, or expert-level judgment [Chen et al., 2025, Starace et al., 2025, Wijk et al., 2025]. Skill- and tool-using agents make that opportunity larger and the governance problem sharper [Yang et al., 2026]. Relaytic-AML responds by making model scores, public claims, handoff packets, and paper assets downstream of local artifacts rather than downstream of a conversation transcript.

3 System Overview

Relaytic-AML is built around one authority rule: the workspace owns the truth. Raw data, licensed benchmark files, run summaries, traces, metric cells, model outputs, tables, figures, and release reports live on disk. Agents may explain, propose, and repair, but their proposals only become evidence when they are materialized as artifacts another human or agent can inspect.

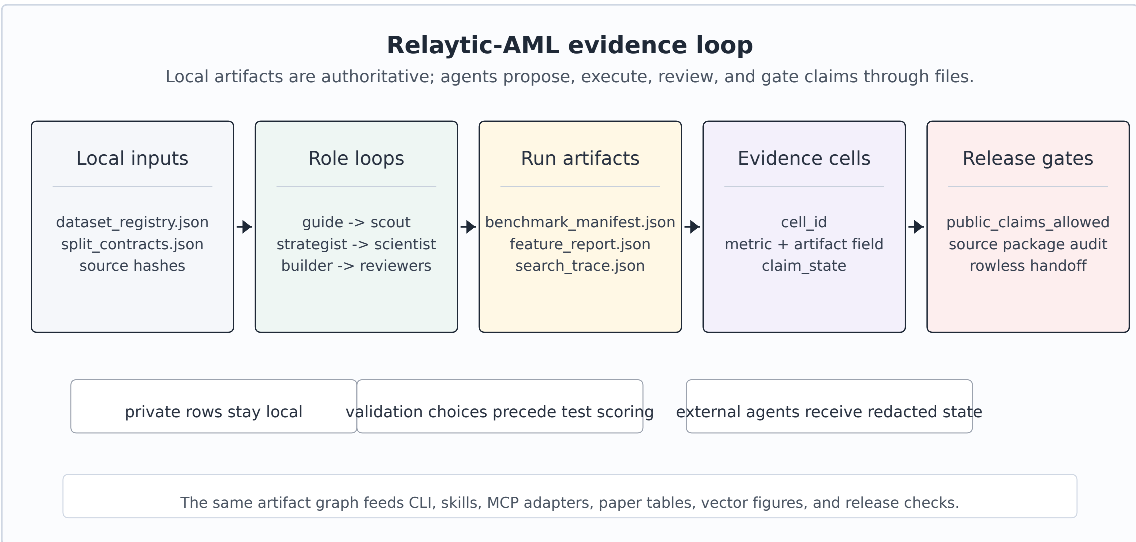

Figure 1: Relaytic-AML local-first architecture: local data and artifacts flow through role-scoped agents into evidence cells, claim gates, and paper/release/handoff surfaces.

Figure 1 summarizes the local evidence loop. Dataset registries and split contracts enter the role-scoped agent runtime. Candidate runs write benchmark manifests, search traces, feature reports, and metric cells. Claim gates read those cells together with release audits and emit only the interpretations that the evidence supports. The same contract feeds the command-line interface, project skills, OpenClaw-style handoff, Claude/Codex skill files, and Model Context Protocol (MCP) adapters.

The external-agent path is rowless by default. Relaytic can export current state, next-action options, artifact shortlists, and safe commands for another model without sending raw transaction rows, secrets, or private local paths. A local LLM can optionally help phrase guidance, but the deterministic guide and evidence artifacts remain the source of truth. That separation matters for private AML work: outside intelligence may help navigate the run, but data residency and claim provenance stay local unless an operator deliberately changes policy.

4 Evidence Cell and Claim-Gate Design

An evidence cell is the unit that makes a paper number auditable. It is not just a metric value. It records the dataset, split, command, artifact field, model or feature budget, leakage posture, operating point, and claim state. That structure gives a reviewer two kinds of visibility: how the number was produced, and what the number is allowed to mean.

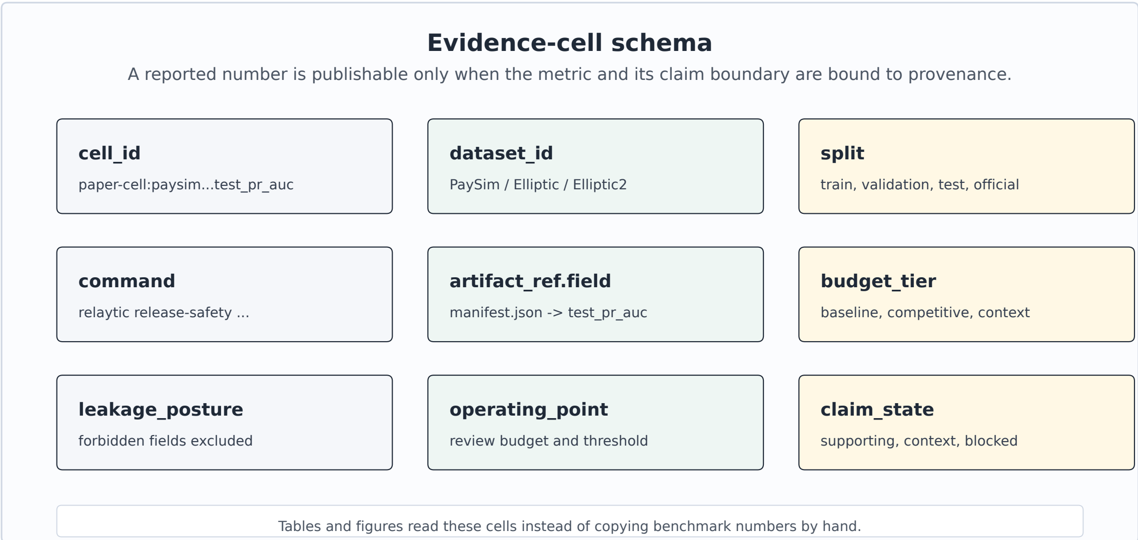

Figure 2: Evidence-cell schema: every reported number carries dataset, split, command, artifact, budget, leakage posture, operating point, metric, and claim state.

Table 1 uses compact publication aliases for readability; the full machine metric-cell identifiers are preserved in the metric-cell audit artifact and generated table comments.

Table 1. Representative evidence cells.

Evidence cell	Dataset	Metric	Value	Split	Evidence	Budget	Claim state
PS-PR	PaySim	test PR-AUC	0.6388	test	PaySim run; manifest	competitive	supporting only
PS-P@B	PaySim	precision at review budget	0.7033	test	PaySim run; manifest	competitive	supporting only
EL-PR	Elliptic	test PR-AUC	0.6688	test	graph run; feature table	competitive	supporting only
EL-P@B	Elliptic	precision at review budget	1	test	graph run; feature table	competitive	supporting only
E2-PRm	Elliptic2	official test PR-AUC mean	0.9432	official test	E2 run; scorecard	competitive context	context only; no contribution
E2-ref	Elliptic2	published reference PR-AUC	0.974	reported reference	E2 run; scorecard	competitive context	context only; no contribution

Algorithm 1 Evidence-cell creation

Input: dataset registry D, split contract S, candidate budget B, run artifacts A
Output: evidence cell c with provenance and claim state
Freeze source posture, license posture, task target, and split contract.
Derive only features allowed by S and record excluded leakage fields.
Run baseline candidates under the declared baseline budget.
Run stronger candidates only within B and select on validation evidence.
Evaluate the selected row once on the fixed test surface.
Write c = dataset, split, command, artifact field, metric, value, budget, leakage posture, operating point, claim state.
Publish c only through tables, figures, and release text that reference the cell identifier.

The claim gate is the second half of the design. It is deliberately conservative. If the evidence cell is incomplete, if a split is leakage-prone, if a metric is only a proxy, or if a stronger interpretation needs a different dataset or study, the gate preserves the evidence but blocks the stronger claim. This is a mechanism, not a disclaimer: it changes what the paper generator and public release surfaces are allowed to say.

Algorithm 2 Claim-gate validation

Input: proposed public claim q, evidence cells C, publishability gates G, limitation matrix L
Output: allowed wording or blocked-claim record
Resolve every evidence cell named by q and require dataset, split, command, artifact, budget, and leakage fields.
Compare the strength of q with source posture, split validity, metric scope, and benchmark role.
If q is exactly supported, emit the bounded wording and the evidence-cell identifiers.
If q is stronger than C and G permit, block q and write the gate reason.
Attach the missing evidence needed to move q from blocked to testable future work.
Keep blocked evidence visible, but route it to limitations or context rather than headline claims.

Figure 3 gives concrete gate behavior. A PaySim row can support a synthetic temporal-fraud result, but not a real-bank AML superiority statement. An Elliptic row can support temporal

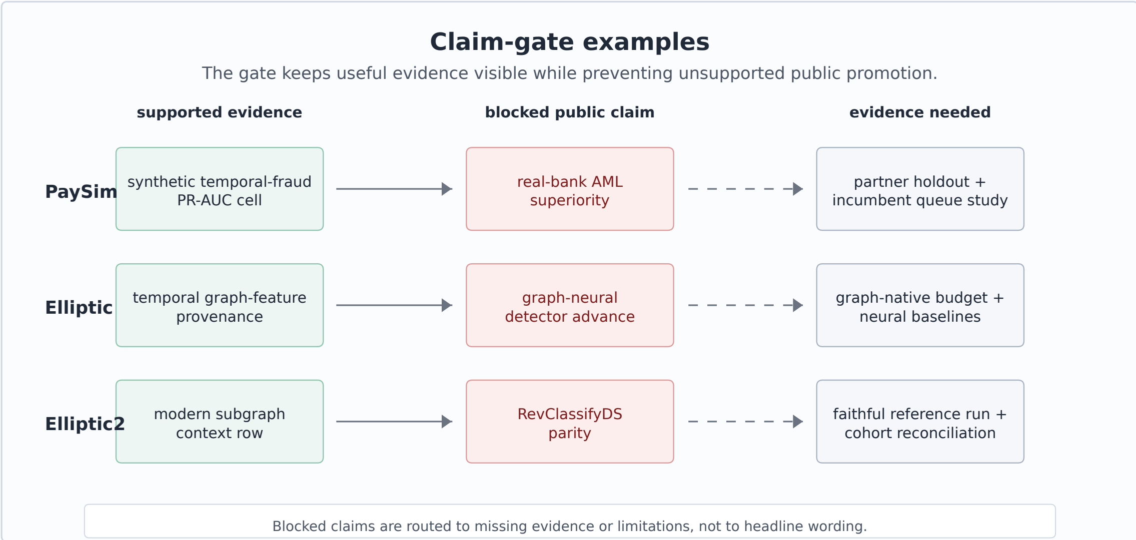

Figure 3: Claim-gate examples: allowed claims, blocked promotions, and evidence needed before stronger public interpretations.

graph-feature provenance, but not graph-neural dominance. An Elliptic2 row can be modern benchmark context while the reference-parity claim remains blocked.

5 Experimental Protocol

The experiments test Relaytic-AML as an evaluation lab. The datasets are separated by claim posture. PaySim is the main empirical demonstration because it is fully local and supports a controlled temporal proxy workflow. Elliptic tests temporal graph provenance and feature-view discipline. Elliptic2 tests whether the system can keep modern-context evidence visible without promoting it to a performance contribution.

Table 2a. Dataset scale and split contracts.

Dataset	Task	Scale and positives	Train / validation / test	Split rule	Source hash
PaySim synthetic mobile-money transaction fraud	Synthetic temporal transaction fraud	6,362,620 rows/nodes; 8213 positives	train: 6,010,937, pos 0.08%; validation: 228,103, pos 0.68%; test: 123,580, pos 1.34%	time step	sha256 prefix 16910f90577b
Elliptic Bitcoin transaction graph	Temporal Bitcoin graph node classification	203,769 rows/nodes; 4545 positives	train: 26,381, pos 10.88%; validation: 8,999, pos 11.53%; test: 11,184, pos 5.69%	graph time	sha256 prefix 93e2e7b2405c plus 2 files
Elliptic2 large Bitcoin subgraph AML dataset	Suspicious-versus-licit subgraph context	122,000 labeled subgraphs; context row	val 11,059, test 11,105; test positives 272	fixed partition	not bundled

Table 2b. Feature and metric policy.

Track	Allowed feature policy	Forbidden or gated inputs	Primary metrics	Claim posture
PaySim	26 decision-time features; balance columns excluded	balance columns, raw account IDs, simulator flags, random row shuffles	PR-AUC, P@k, R@budget	supporting-only
Elliptic	structural same snapshot only; source provided flattened features; source features plus structural snapshot	future-to-train edges and random node splits across time	PR-AUC, P@k, fixed-FPR recall	supporting-only
Elliptic2	pooled moments counts; raw subgraphs local-gated	claim use gated by reference parity and local data availability	PR-AUC, P@k, fixed-FPR recall	blocked

Tables 2a and 2b record the context a reader needs before interpreting any metric. The positive rates show why precision-recall area under the curve (PR-AUC) is the primary score. These are rare-event tasks where receiver-operating-characteristic area under the curve can look strong while the review queue remains poor. The split rules are equally important: PaySim is split by chronological simulator step, Elliptic by time-step graph windows, and Elliptic2 by the official/context partitions recorded in the local evidence pack.

Table 3. Model families and search budgets.

Track	Model family scope	Feature family scope	Budget	Selection rule	Seeds
PaySim	tree ensembles and gradient boosting; fixed-rule baseline retained	amount, type, time, train-only thresholds, shifted destination history	14 probes; 5 full finalists	validation PR-AUC only; Platt calibration	42
Elliptic	tree ensembles and gradient boosting over frozen feature views	source anonymized node features, same-snapshot graph features, combined view	16 validation trials; LightGBM selected	validation PR-AUC within declared feature views; validation-only calibration and threshold	42
Elliptic2	pooled-moment LightGBM context candidate	348 pooled subgraph moment/count features	3 repeated seeds; context row only	official partition plus content-hash robustness; reference parity gate remains blocked	11, 42, 73

Table 3 records the modeling effort without presenting the paper as a hyperparameter leaderboard. PaySim uses a two-stage competitive budget: probes over a seeded train-only sample, five full-training finalists, validation-only calibration, and one fixed test evaluation for the selected finalist. Elliptic uses a graph-feature budget over source-provided anonymized features, same-snapshot structural features, and their combination. Elliptic2 uses repeated pooled-moment LightGBM context rows with seeds 11, 42, and 73, but this row is not a Relaytic performance contribution because reference-protocol parity is unresolved.

The feature policy is strictest for PaySim because simulator balance columns can leak post-event state. Relaytic excludes those balance fields, raw origin and destination identifiers as model features, and simulator flags. It allows row-local amount, type, and time features, train-only thresholds, and destination history shifted before each step. The isolated contribution of destination-history features has not been measured as a separate test row, so it is not reported as a main result.

6 Results

Table 4. PaySim modeling path.

Stage	Model or contract	Change	Test PR-AUC	Claim use
P4 reference	SGD logistic baseline	source-safe starting point	0.2159	not a paper claim
P6 baseline	Extra Trees baseline	leakage-safe feature set	0.3313	baseline only
Competitive search	XGBoost probe	26 allowed features; best validation PR-AUC 0.5944	test hidden until selected finalist	search evidence only
Final model	Extra Trees with Platt calibration	validation-selected finalist evaluated once on test	0.6388	supporting synthetic temporal-proxy claim

PaySim is the clearest modeling result in the current evidence pack. The earliest reference row was PR-AUC 0.2159. The later leakage-safe baseline improved to 0.3313, and the competitive selected Extra Trees model reached fixed-test PR-AUC 0.6388 and ROC-AUC 0.9683. The improvement is meaningful inside the synthetic temporal-fraud contract because it follows a visible sequence: balance fields were excluded, prior-step destination history was added without raw account encoding, candidates were selected by validation PR-AUC, and calibration and thresholding used validation-only partitions. The allowed claim is precise: Relaytic-AML can produce a stronger, leakage-audited PaySim temporal-proxy row under a declared budget. It cannot turn that row into evidence of real-bank AML superiority.

The review-budget metrics sharpen the interpretation. At the selected PaySim review budget, precision is 0.7033 and recall is 0.4716. The top of the queue is much richer than prevalence, but a large share of fraud remains outside the reviewed set. That is a useful operating result for an evaluation lab because it connects ranking quality to analyst capacity instead of treating PR-AUC as the whole story.

Elliptic is a different kind of evidence. The validation-selected source-plus-structural LightGBM row reports test PR-AUC 0.6688, with review-budget precision 1 and recall 0.0566. The result supports temporal graph-feature provenance and operating-point reporting. It also reveals a limitation: the current graph-structure-only floor is weak, and the final row is heavily influenced by source-provided anonymized features. Relaytic can therefore claim graph-aware evidence handling, not a new graph-native detector.

Elliptic2 is intentionally framed as context. The repeated official-partition context row reports PR-AUC 0.9432 +/- 0.0009, and the content-hash robustness partition reports mean PR-AUC 0.9297. Those are strong absolute values, but the recorded RevClassifyDS reference is 0.974, and the local parity gate records unresolved cohort and reference-execution issues. The scientific result is that Relaytic keeps the context row available while blocking a parity or superiority interpretation.

Figure 4 separates ranking metrics from operating-point metrics. PR-AUC summarizes ranking quality under rare-event imbalance. Precision and recall at the selected review budget describe what the top of an analyst queue would contain under the paper's fixed policy. Keeping those views together, but visually separated, is important because a useful top queue can still leave many positives unreviewed.

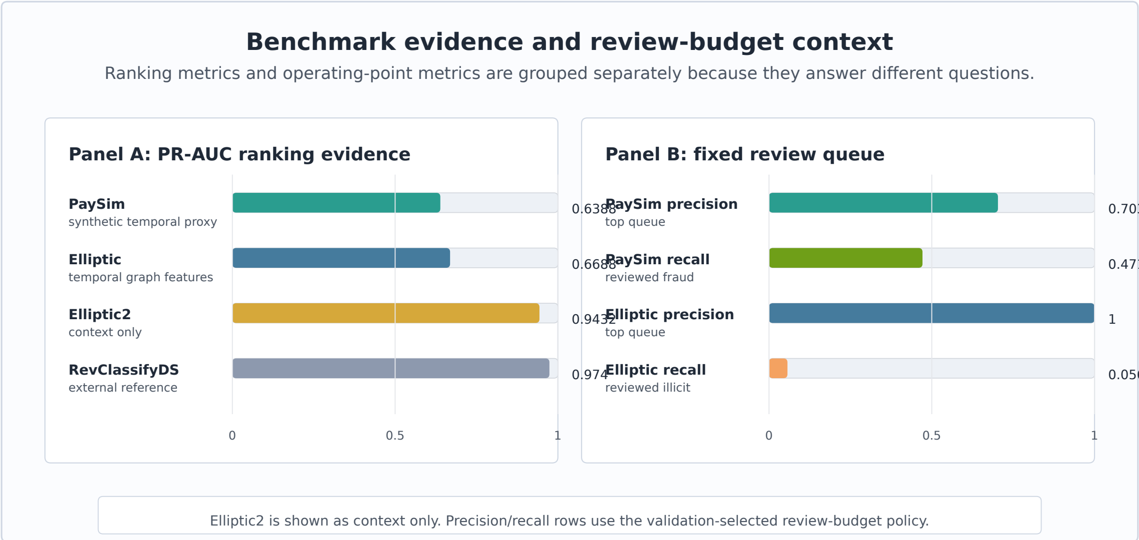

Figure 4: Benchmark and review-budget evidence: PR-AUC is shown beside precision and recall at the bounded review queue instead of being interpreted alone.

7 System Evaluation

The system claim is evaluated through deterministic reader and agent tasks. These checks ask whether a reviewer can navigate from the README to the paper evidence, trace PaySim metric provenance, compare baseline and competitive budgets, keep Elliptic2 as context rather than a contribution, export rowless state for an external agent, recover an interrupted run, and block over-strong public claims. This is narrower than a human-subject usability study, but it directly tests the infrastructure claim made by the paper.

Table 5. System audit matrix.

Check	Command or test	Evidence	Pass criterion	Observed result
Metric provenance	metric provenance	PaySim selected PR-AUC cell	required fields present	pass; fields 13/13
Budget comparison	budget comparability	PaySim baseline and competitive cells	same dataset, split doctrine, and metric	pass; 0.3313 to 0.6388
PaySim claim boundary	claim-boundary check	PaySim publishability row	supporting=true; hard/headline=false	pass; supporting only
Elliptic2 firewall	reference-parity firewall	Elliptic2 publishability row	context visible; parity blocked	pass; context only
Rowless handoff	handoff recovery	agent handoff report	state, tools, next action; no rows	pass; rowless handoff
Interrupted recovery	no-lost-user recovery	guide recovery report	stage, shortlist, next action exposed	pass; partial recovery
Claim fail-closed	claim-gate cases	blocked-claim case studies	unsupported wording blocked	pass; case gates pass

Table 5 reports the audit matrix behind the system claim. The important result is not that a script returned a generic pass flag. It is that each checked behavior has an observed signal: 13 of 13 provenance fields are present for the PaySim metric cell, baseline and competitive budgets are comparable under the same contract, Elliptic2 is firewalled as context, rowless handoff exposes no raw rows, and interrupted-run recovery surfaces state, missing evidence, and next actions.

Table 6. Blocked claim examples.

Proposed invalid claim	Gate reason	Missing evidence
Relaytic-AML is superior for real-bank AML detection.	PaySim is synthetic and the current evidence is a temporal proxy result.	Partner or bank-approved holdout, incumbent comparison, analyst-review protocol, and legal release gate.
Relaytic-AML reaches RevClassifyDS parity on Elliptic2.	The context row is below the recorded reference and reference-protocol parity remains unresolved.	Faithful reference execution, cohort reconciliation, resource budget, and repeated parity report.
Relaytic-AML is a graph-neural detector advance.	Elliptic evidence uses graph-feature provenance and tree/boosting baselines, not a new graph-neural family.	Graph-native release budget, neural baselines, repeated seeds, and graph-specific ablations.

Table 6 shows how the gate behaves on concrete over-strong claims. The blocked claims are not hidden. They are converted into missing-evidence records, which makes future work actionable while keeping the current paper below its evidence boundary.

Table 7. Rowless handoff and interrupted-run recovery examples.

Scenario	Input state	Exported fields	Redacted fields	Observed signal
External-agent handoff	partial run with available guide state	state summary, action options, starter questions, tool contract, artifact shortlist	raw transaction rows, credentials, private paths, raw source files	raw_rows=False; redactions=8; blocked_fields=6
Safe next action	external model asked what to do next	six next actions, six starter questions, command options	unredacted local paths and data rows	actions=6; starter_questions=6
Interrupted-run recovery	operator returns to partial run without artifact literacy	current state, missing evidence count, canonical artifact shortlist, context-export command	raw benchmark data and private machine paths	state=partial_run; missing=8; actions=6

Table 7 gives the practical external-agent story. A second model can receive state, commands, artifacts, and starter questions, while raw rows remain redacted together with private machine paths. The same mechanism helps an inexperienced or interrupted user recover the next action without knowing which internal artifact to inspect first.

8 Limitations and Threats to Validity

PaySim is synthetic. It is useful for controlled temporal fraud experiments, but it is not evidence of bank-scale AML superiority. The simulator has known simplifications, and the current result should be interpreted as a leakage-audited proxy result. The destination-history feature contract is present, but the isolated destination-history ablation is not in the current evidence pack, so no separate result is claimed for that feature family.

Public blockchain data is also not the same as bank AML. Elliptic provides a valuable temporal graph task, but unknown labels, anonymized features, and public-chain behavior limit direct operational interpretation. Elliptic2 is modern and highly relevant, but the current local evidence does not satisfy the stronger reference-parity conditions needed for a performance contribution against RevClassifyDS.

The deterministic system checks are not a substitute for a human usability study. They show that artifacts, redactions, claim gates, and recovery surfaces are present and internally consistent. They do not measure analyst time, production incident rates, organizational adoption, or real investigation quality. Future work should test stronger release budgets, repeated runs, private or partner-approved holdouts, same-queue incumbent comparisons, and graph-native families under the same evidence-cell discipline.

The system is intentionally local-first, which creates a tradeoff. Privacy and provenance improve because raw rows stay local, but external reviewers cannot rerun licensed or private data without obtaining it themselves. The paper handles that by publishing commands, hashes where allowed, generated artifacts, and claim boundaries, but a fully independent reproduction of every heavy benchmark still depends on legal access to the source datasets.

9 Reproducibility

The repository is larger than this AML paper. Relaytic is the general local-first inference lab and public package; Relaytic-AML is the focused AML edition used here for the manuscript. A reader should start with the README and this paper. Development-control files record the build history, but they are not required to understand the paper claims.

Table 8. Reproducibility contract.

Component	Command	Expected output	Environment or data dependency	Hash or seed record
Paper artifacts	paper-release; paper-arxiv-source	Markdown draft, arXiv main.tex, vector figures	Python >=3.10; CI matrix covers 3.10 and 3.11; full extra installs numpy, pandas, scikit-learn, matplotlib	deterministic generators; source hashes in manifest
PaySim benchmark	release-safety paysim-competitive –budget-tier competitive –run-optional –format json	PaySim selected PR-AUC cell	local Kaggle PaySim file required	sha256 prefix 16910f90577b
Elliptic benchmark	release-safety graph-baselines –budget-tier competitive –run-optional –format json	Elliptic selected PR-AUC cell	local Kaggle Elliptic files required	sha256 prefix 93e2e7b2405c plus 2 files
System evaluation	release-safety paper-system-eval –format json	task, handoff, recovery, and claim-gate reports	repo-local deterministic fixtures	all required system-evaluation tasks pass in current evidence pack

Minimal regeneration commands are shown below.

Windows PowerShell:

```
py -3.11 -m pip install -e ".[full]"
py -3.11 -m relaytic.ui.cli release-safety paper-system-eval --format json
py -3.11 -m relaytic.ui.cli release-safety paper-release --format json
py -3.11 -m relaytic.ui.cli release-safety paper-arxiv-source --format json
py -3.11 -m pytest ‘
  tests/test_paper_track_p13.py ‘
  tests/test_paper_track_p14.py ‘
  tests/test_paper_track_p15.py -q
```

macOS/Linux:

```
python3 -m pip install -e ".[full]"
python3 -m relaytic.ui.cli release-safety paper-system-eval --format json
python3 -m relaytic.ui.cli release-safety paper-release --format json
python3 -m relaytic.ui.cli release-safety paper-arxiv-source --format json
python3 -m pytest \
  tests/test_paper_track_p13.py \
  tests/test_paper_track_p14.py \
  tests/test_paper_track_p15.py -q
```

Raw benchmark data is not committed. PaySim and Elliptic require local downloads and are referenced through registry artifacts, split reports, hashes, and command ledgers. Elliptic2 remains context-only in this paper because the stronger reference-parity conditions are not satisfied locally. Clean clones can reproduce the paper-generation checks and repo-local public fixtures; full benchmark regeneration requires the locally licensed datasets named in the README.

LLM tools assisted with drafting, editing, repository inspection, and consistency checks. They are not authors. The evidence cells, code, figures, tables, limitations, and final claims remain the author's responsibility.

10 Conclusion

Relaytic-AML shows how an agent-assisted AML evaluation lab can be built around local evidence rather than conversational memory. The system keeps data posture, temporal and graph split validity, leakage controls, model budgets, review-budget operating points, rowless handoff, and public claims inside one artifact record. The PaySim, Elliptic, and Elliptic2 rows are useful because they demonstrate that architecture under realistic forms of pressure, including rare events, graph provenance, modern benchmark context, and blocked claims.

The strongest claim supported today is architectural: Relaytic-AML can make AML experiments easier to inspect, easier to challenge, safer to hand off to another agent, and harder to overstate. Stronger detector and business-value claims require stronger evidence. That is the boundary the system is designed to preserve.

References

Claudio Bellei, Muhua Xu, Ross Phillips, Tom Robinson, Mark Weber, Tim Kaler, Charles E. Leiserson, Arvind, and Jie Chen. The shape of money laundering: Subgraph representation learning on the blockchain with the elliptic2 dataset, 2024. URL `https://arxiv.org/abs/2404.19109`.

Hui Chen, Miao Xiong, Yujie Lu, Wei Han, Ailin Deng, Yufei He, Jiaying Wu, Yibo Li, Yue Liu, and Bryan Hooi. Mlr-bench: Evaluating ai agents on open-ended machine learning research, 2025. URL `https://arxiv.org/abs/2505.19955`.

Keyang Chen, Mingxuan Jiang, Yongsheng Zhao, Zeping Li, Zaiyuan Chen, Weiqi Luo, Zhixin Li, Sen Liu, Yinan Jing, Guangnan Ye, Xihong Wu, and Hongfeng Chai. Transxion: A high-fidelity graph benchmark for realistic anti-money laundering, 2026. URL `https://arxiv.org/abs/2604.17420`.

Bruno Deprez, Wei Wei, Wouter Verbeke, Bart Baesens, Kevin Mets, and Tim Verdonck. Advances in continual graph learning for anti-money laundering systems: A comprehensive review, 2025. URL `https://arxiv.org/abs/2503.24259`.

Federal Financial Institutions Examination Council. Bsa/aml examination manual: Appendix f, money laundering and terrorist financing red flags, 2026. URL `https://bsaaml.ffiec.gov/manual/Appendices/07`.

Financial Action Task Force. Virtual assets red flag indicators of money laundering and terrorist financing, 2020. URL `https://www.fatf-gafi.org/en/publications/Methodsandtrends/Virtual-assets-red-flag-indicators.html`.

Financial Crimes Enforcement Network. Alerts, advisories, notices, bulletins, and fact sheets, 2026. URL `https://www.fincen.gov/resources/advisoriesbulletinsfact-sheets`.

Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daume III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021. doi: 10.1145/3458723. URL `https://arxiv.org/abs/1803.09010`.

Edgar Alonso Lopez-Rojas, Ahmad Elmir, and Stefan Axelsson. Paysim: A financial mobile money simulator for fraud detection. In *Proceedings of the 28th European Modeling and Simulation Symposium*, 2016. URL `https://www.msc-les.org/proceedings/emss/2016/EMSS2016_249.pdf`.

Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229, 2019. doi: 10.1145/3287560.3287596. URL `https://arxiv.org/abs/1810.03993`.

Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Lariviere, Alina Beygelzimer, Florence d'Alche Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research. *Journal of Machine Learning Research*, 22(164):1–20, 2021. URL `https://www.jmlr.org/papers/v22/20-303.html`.

Chung-Hoo Poon, James Kwok, Calvin Chow, and Jang-Hyeon Choi. Linemvgnn: Anti-money laundering with line-graph-assisted multi-view graph neural networks, 2026. URL `https://arxiv.org/abs/2603.23584`.

Kiwhan Song, Mohamed Ali Dhraief, Muhua Xu, Locke Cai, Xuhao Chen, Arvind, and Jie Chen. Identifying money laundering subgraphs on the blockchain. In *Proceedings of the 5th ACM International Conference on AI in Finance*, 2024. doi: 10.1145/3677052.3698635. URL `https://arxiv.org/abs/2410.08394`.

Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal Patwardhan. Paperbench: Evaluating ai's ability to replicate ai research, 2025. URL `https://arxiv.org/abs/2504.01848`.

Haseeb Tariq and Marwan Hassani. Extracting money laundering transactions from quasi-temporal graph representation, 2026. URL `https://arxiv.org/abs/2604.02899`.

Mark Weber, Giacomo Domeniconi, Jie Chen, Daniel Karl I. Weidele, Claudio Bellei, Tom Robinson, and Charles E. Leiserson. Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics, 2019. URL `https://arxiv.org/abs/1908.02591`.

Hjalmar Wijk, Tao Roa Lin, Joel Becker, Sami Jawhar, Neev Parikh, T. Broadley, Lawrence Chan, Michael Chen, Joshua M. Clymer, Jai Dhyani, Elena Ericheva, Katharyn Garcia, Brian Goodrich, Nikola Jurkovic, Megan Kinniment, Aron Lajko, Seraphina Nix, Lucas Jun Koba Sato, William Saunders, Maksym Taran, Ben West, and Elizabeth Barnes. Re-bench: Evaluating frontier ai r&d capabilities of language model agents against human experts. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 66772–66832, 2025. URL `https://proceedings.mlr.press/v267/wijk25a.html`.

Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. Skillopt: Executive strategy for self-evolving agent skills, 2026. URL `https://arxiv.org/abs/2605.23904`.

Haojie Ye, Arjun Laxman, Yichao Yuan, Krisztian Flautner, and Nishil Talati. Blazingaml: High-throughput anti-money laundering (aml) via multi-stage graph mining, 2026. URL `https://arxiv.org/abs/2604.12241`.